

La información biológica y su representación en Python

BC-CH01 – Biología Computacional

Edición 2 · 2026-09-03

Pregunta del tema. Una cadena de texto con cuatro letras distintas puede ser un fichero cualquiera o el programa completo de un ser vivo. ¿Qué convierte una secuencia de ADN en información, y qué operaciones sobre ella significan algo biológico y no solo sintáctico?

Producto final. Un módulo propio en Python que calcule contenido GC, complemento reverso, transcripción y perfiles por ventana, con una guarda que rechace lo que no es ADN.

Mapa del tema

1. Por qué la biología se dejó leer como información	1
2. El Dogma Central y lo que lo rompe	2
3. Ácidos nucleicos: el soporte del texto	3
4. Proteínas: la maquinaria	4
5. Carbohidratos: energía, andamio y etiqueta	6
6. Lípidos: la frontera de la célula	7
7. Qué hace Python cuando guarda una secuencia	8
8. Cuatro operaciones sobre secuencias	10
9. Un módulo propio de secuencias	12
10. Un genoma que no cabía en sí mismo	13
11. Tres errores que va a cometer	13
12. Síntesis y tablas de diagnóstico	14
13. Glosario	14
14. Con qué sigue este capítulo	15
15. Fuentes y reproducibilidad	15

Cómo usar este tema

Este capítulo tiene dos mitades que conviene no separar. La primera es química: qué son las moléculas que vamos a representar y qué propiedades tuyas importan. La segunda es computacional: cómo guarda Python esa información y qué cuesta cada operación. Están juntas porque las decisiones de la segunda solo se entienden desde la primera. Que una secuencia se escriba siempre de 5' a 3' no es una convención de formato: es la razón por la que el complemento reverso lleva una inversión y no solo un cambio de letras.

Al terminar sabrá: explicar por qué una secuencia de ADN es información y no solo una cadena; nombrar las cuatro familias de biomoléculas y decir qué representa cada una en un programa; predecir qué ocurre en memoria cuando modifica una lista de secuencias; calcular a mano el contenido GC y el complemento reverso, y comprobar su cálculo con el oráculo del capítulo; y escribir funciones que rechacen una entrada inválida en lugar de producir un número sin sentido.

La ruta **esencial** son las cuatro familias de biomoléculas, el modelo de memoria de Python, las cuatro operaciones sobre secuencias y la ventana deslizante en tiempo $O(N)$. La ruta de **estudio** añade la estereoquímica de los azúcares, la física de la bicapa lipídica, el caso de $\Phi X174$, la actividad de depuración y el anexo. Antes de ejecutar cada orden, escriba lo que espera obtener: el capítulo está construido sobre ese contraste.

1 Por qué la biología se dejó leer como información

ESENCIAL Durante casi todo el siglo XIX la biología describía. Clasificaba organismos por su forma, seguía el balance químico de una reacción, medía. Era una ciencia de sustancias y de procesos, no de mensajes. Que hoy hablemos de «leer» un genoma, de «editar» o de «programar» una célula procede de tres trabajos que en su momento no tenían nada que ver entre sí.

El primero es de Turing, en 1936 [7]. Turing no estudiaba células: quería saber qué problemas puede resolver un procedimiento mecánico. Para responderlo describió una máquina que lee y escribe símbolos discretos sobre una cinta, y demostró que ese artefacto tan pobre basta para cualquier cómputo. La consecuencia que nos interesa es indirecta pero decisiva: si un proceso manipula símbolos discretos según reglas fijas, es computación, sea cual sea el material del que esté hecho.

El segundo es de Shannon, en 1948 [6]. Shannon trabajaba en telefonía y necesitaba medir cuánta información lleva un mensaje. Su respuesta, la cantidad de incertidumbre que el mensaje elimina, tiene una propiedad extraña: no depende del soporte. La misma información viaja por un cable, por el aire o por una molécula. Nada obliga a que el soporte de un mensaje sea electrónico.

El tercero es de Watson, Crick y Franklin, en 1953 [9]. La doble hélice mostró que el material hereditario es una secuencia lineal de cuatro piezas químicas distintas que se repiten sin patrón periódico. Es decir: exactamente el tipo de objeto del que hablaban los dos primeros.

Puestos uno junto a otro, los tres dicen algo que ninguno decía por separado. Una célula almacena instrucciones en un polímero, las copia, las transmite a su descendencia y las ejecuta. Eso es procesamiento de información, con un soporte químico en lugar de electrónico.

Atención. Conviene tomarse la metáfora con cuidado. Que el ADN se comporte como un texto no significa que la célula sea un ordenador. Un ordenador separa el programa de la máquina que lo ejecuta; una célula no siempre lo hace, como veremos en cuanto miremos las excepciones del Dogma Central. La representación digital es una herramienta que funciona muy bien para casi todo lo que haremos en este libro, y este capítulo se ocupa también de señalar dónde deja de funcionar.

De ahí sale el programa de trabajo de la biología computacional, que en la práctica exige tres cosas a la vez. Hay que entender el sustrato: qué fuerzas y qué restricciones termodinámicas gobiernan las moléculas, porque son las que hacen que unas operaciones tengan sentido y otras no. Hay que formalizar: convertir el fenómeno en un objeto discreto, una cadena, un grafo, una distribución. Y hay que implementar de forma que el cálculo termine, lo que con genomas de 10^6 a 10^{10} bases no es una cuestión de estilo sino de viabilidad.

La biología computacional representa la información biológica mediante objetos discretos y algoritmos deterministas cuyo coste puede acotarse.

2 El Dogma Central y lo que lo rompe

ESENCIAL Si la célula guarda instrucciones, ¿por dónde circulan? La respuesta canónica la enunció Crick en 1958 y consta de tres procesos, que la figura 1 resume.

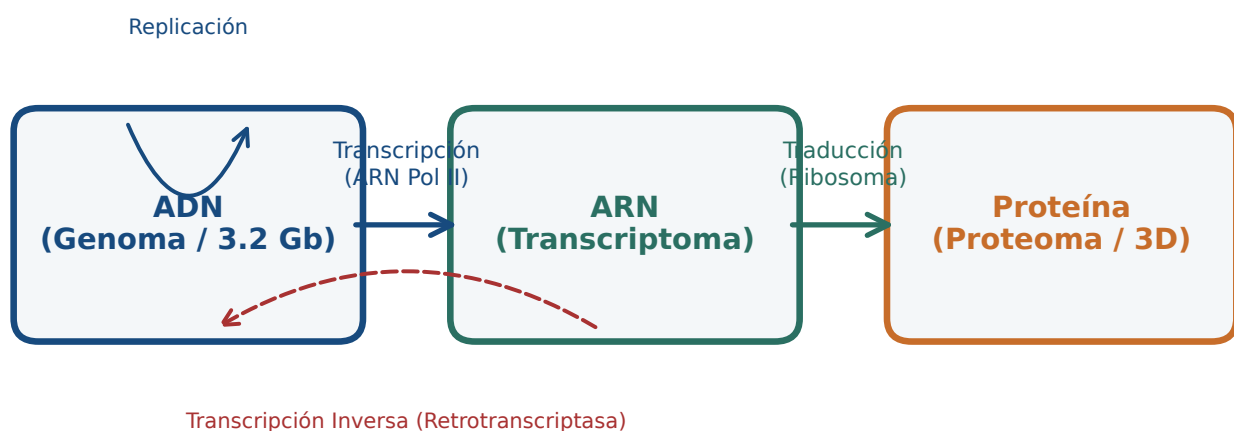


Figura 1: Los tres flujos del Dogma Central y dos de sus excepciones. Figura original generada por `verification/make_figures.py`.

- **Replicación**, de ADN a ADN. Es la copia del archivo completo antes de cada división celular.

- **Transcripción**, de ADN a ARN. Es la lectura de un fragmento concreto, un gen, para obtener una copia de trabajo.
- **Traducción**, de ARN a proteína. El ribosoma ejecuta esa copia y convierte una secuencia de nucleótidos en una secuencia de aminoácidos.

Enunciado así, el dogma reparte los papeles con nitidez: los ácidos nucleicos guardan y transmiten, las proteínas ejecutan. El archivo por un lado, la maquinaria por otro. Es un reparto cómodo, y es el que justifica que este libro represente las secuencias como cadenas de texto.

Cuatro fenómenos conocidos desdibujan ese reparto. No son curiosidades de examen: son justamente los casos donde la representación que vamos a usar se queda corta, y por eso vale la pena conocerlos desde el principio.

- La **transcripción inversa**, de ARN a ADN, que realizan los retrovirus y las retrotransposasas. La flecha va hacia atrás.
- La **replicación del ARN**, en virus cuyo genoma es ARN. El archivo nunca pasa por ADN.
- Las **ribozimas**, moléculas de ARN con actividad catalítica. El propio centro peptidil-transferasa del ribosoma, el que forma el enlace peptídico, es una de ellas. Aquí el supuesto portador de información es la maquinaria.
- Los **priones**, proteínas capaces de imponer su conformación anómala a otras copias de la misma proteína. La información se propaga sin ácido nucleico de por medio, en el plegamiento y no en la secuencia.

Atención. Las dos últimas importan especialmente para lo que sigue. Un prión demuestra que hay información biológica que la secuencia no captura: dos moléculas con la misma secuencia y distinto plegamiento se comportan de forma distinta, y ninguna cadena de texto distingue una de otra. Esa limitación de la representación reaparecerá en los capítulos de estructura de proteínas, donde la secuencia deja de bastar.

3 Ácidos nucleicos: el soporte del texto

ESENCIAL. La materia viva se organiza en cuatro grandes familias de moléculas. Cada una tiene una lógica propia, y esa lógica determina cómo la representamos en un programa [1]. Empezamos por la que este libro usa más.

Las cuatro familias biomoleculares, ácidos nucleicos, proteínas, carbohidratos y lípidos, tienen propiedades estructurales y funcionales distintas que condicionan su representación computacional.

Los ácidos nucleicos, ADN y ARN, son los portadores del material hereditario en los tres dominios de la vida y en los virus. Son polímeros lineales sin ramificar, hechos de **nucleótidos**, y cada nucleótido tiene tres piezas unidas covalentemente:

1. Un azúcar de cinco carbonos en forma de anillo: **2-desoxi-D-ribosa** en el ADN, que carece del hidroxilo en la posición 2', y **D-ribosa** en el ARN, que sí lo tiene. Ese hidroxilo de más es la razón química de que el ARN sea mucho más frágil: hace la molécula más reactiva y sensible a la hidrólisis. Cuando en el laboratorio el ARN se degrada y el ADN no, el culpable es ese único grupo.
2. Un grupo fosfato unido al carbono 5' del azúcar.
3. Una base nitrogenada unida al carbono 1' por un enlace β -N-glucosídico.

Las bases se reparten en dos familias por su forma: **purinas**, con dos anillos fusionados, que son la adenina (A) y la guanina (G); y **pirimidinas**, con un solo anillo, que son la citosina (C), la timina (T, solo en el ADN) y el uracilo (U, solo en el ARN). Que una purina se empareje siempre con una pirimidina no es casualidad: es lo que mantiene constante la anchura de la doble hélice.

Un nucleótido consta de fosfato, pentosa y base nitrogenada; el ADN bicatenario con desoxirribosa y timina difiere del ARN monocatenario con ribosa y uracilo.

3.1 Por qué una secuencia tiene dirección

Los nucleótidos se enlazan mediante **enlaces fosfodiéster**: un fosfato une el carbono 3' del nucleótido anterior con el carbono 5' del siguiente. Como los dos extremos que se unen son químicamente distintos, la cadena resultante no es simétrica. Tiene un **extremo 5'**, que termina en fosfato, y un **extremo 3'**, que termina en un hidroxilo libre. Ese hidroxilo del extremo 3' es el punto donde las polimerasas añaden el nucleótido siguiente, y por eso toda síntesis de ADN avanza en el sentido 5' → 3' y nunca al revés.

De ahí sale la convención que gobierna todo lo que haremos:

Concepto. Toda secuencia de ácido nucleico se escribe, se almacena y se procesa en sentido 5' → 3', salvo que se declare explícitamente lo contrario. Cuando alguien le entrega la cadena ATGC sin más aclaración, le está entregando 5'-ATGC-3'. Esta convención es la razón de que más adelante el complemento reverso lleve una inversión: sin ella, el resultado estaría escrito al revés de como todo el mundo lo lee.

3.2 El apareamiento de bases y la fuerza que lo mantiene

En el ADN de doble cadena, las dos hebras corren **antiparalelas**, una en sentido 5' → 3' y la otra en sentido 3' → 5', y se enrollan en una hélice dextrógira de unos 2,0 nm de diámetro. Lo que mantiene enfrentadas las bases correctas son puentes de hidrógeno:

- **Adenina con timina:** dos puentes de hidrógeno, entre N1 y H-N3, y entre N6-H y O4.
- **Guanina con citosina:** tres puentes de hidrógeno, entre O6 y H-N4, entre N1-H y N3, y entre N2-H y O2.

Los tres puentes del par G-C estabilizan más que los dos del par A-T, y por eso una región rica en G y C funde a mayor temperatura. Pero conviene no confundir estos puentes con enlaces covalentes, ni escribirlos como si lo fueran.

Atención. Un puente de hidrógeno es una interacción *no* covalente, entre uno y dos órdenes de magnitud más débil que el enlace covalente que une los átomos dentro de cada base. Esa debilidad no es un defecto: es exactamente lo que permite que la doble hélice se abra y se cierre durante la replicación y la transcripción sin romper ninguna molécula. Si los pares de bases estuvieran unidos covalentemente, copiar el genoma exigiría romper y rehacer enlaces químicos, y la célula no podría hacerlo miles de veces al día.

Hay un segundo punto donde es fácil equivocarse. Los valores de energía que se manejan en la práctica, del orden de -1 a -2 kcal/mol para pares A-T y de -2,5 a -3,5 para pares G-C, no son la energía de los puentes de hidrógeno de un par aislado. Corresponden a pares de bases *contiguos* e incluyen el apilamiento aromático entre bases vecinas, que aporta una parte sustancial de la estabilidad. Por eso el modelo estándar para predecir la temperatura de fusión de un oligonucleótido se llama de **vecino más próximo**: mira parejas de pares, no pares sueltos [5].

Concepto. Retenga la consecuencia práctica: el contenido GC de una secuencia predice bastante bien su estabilidad, pero no la determina. Dos secuencias con el mismo contenido GC y distinto orden de bases funden a temperaturas distintas, porque el apilamiento depende de qué base sigue a cuál. El capítulo sobre diseño de cebadores retoma esta idea con el cálculo completo.

4 Proteínas: la maquinaria

Si los ácidos nucleicos son el archivo, las **proteínas** son quien hace el trabajo. Catalizan reacciones, transportan moléculas a través de membranas, reciben señales, mueven la célula, la sostienen y reconocen lo que es propio y lo que no.

Una proteína es un polímero lineal de **aminoácidos** unidos por enlaces peptídicos. Los aminoácidos que la vida usa para construirlas son veinte, y todos comparten el mismo esqueleto: un carbono central, el C α , con cuatro sustituyentes, un hidrógeno, un grupo carboxilo (-COOH), un grupo amino (-NH₂) y una cadena lateral variable que se abrevia -R. Toda la variedad funcional de las proteínas sale de esa cuarta posición.

Los veinte aminoácidos estándar comparten un carbono alfa con grupos amino y carboxilo y se diferencian por una cadena lateral que determina si son no polares, polares, ácidos o básicos.

4.1 Todos son L, y eso quiere decir algo concreto

Con la excepción de la glicina, el C α tiene cuatro sustituyentes distintos y es por tanto un centro quiral: la molécula y su imagen especular no son superponibles. Existen dos configuraciones posibles, L y D, y las proteínas de todos los seres vivos conocidos usan solo la L.

Concepto. Que un aminoácido sea de la serie L no es una etiqueta arbitraria sino una descripción geométrica. En la proyección de Fischer, con el carboxilo dibujado arriba y la cadena lateral abajo, un aminoácido L tiene el grupo amino a la izquierda, y uno D lo tiene a la derecha. Esa homoquiralidad es una de las regularidades más llamativas de la vida terrestre. Los aminoácidos D no son imposibles: aparecen en las paredes celulares bacterianas y en algunos péptidos antibióticos, precisamente donde interesa que las enzimas del hospedador no los reconozcan.

4.2 Carga eléctrica y punto isoelectrónico

ESTUDIO A pH fisiológico, entre 7,2 y 7,4, un aminoácido libre no es neutro en sus dos extremos: el carboxilo ha perdido su protón ($-\text{COO}^-$, con $\text{p}K_{a1}$ entre 1,8 y 2,4) y el amino lo ha ganado ($-\text{NH}_3^+$, con $\text{p}K_{a2}$ entre 8,8 y 10,5). La molécula tiene dos cargas opuestas y carga neta cero. A esa forma se la llama **zwitterión**.

El **punto isoelectrónico** (pI) es el pH al que la carga neta media es exactamente cero. Para un aminoácido sin cadena lateral ionizable se calcula promediando sus dos $\text{p}K_a$:

$$\text{pI} = \frac{\text{p}K_{a1} + \text{p}K_{a2}}{2}$$

Cuando la cadena lateral también se ioniza hay tres valores, y se promedian los dos que flanquean la forma neutra. Para los ácidos (Asp, Glu) el pI queda entre 2,8 y 3,2, de modo que a pH fisiológico llevan carga -1 . Para los básicos (Lys, Arg) queda entre 9,7 y 10,8, y a pH fisiológico llevan carga $+1$. De ahí sale una regla útil: a pH fisiológico, la superficie de una proteína está dominada por los residuos cargados, y su carga neta decide cómo se comporta en una columna de intercambio iónico o en un gel.

4.3 Las cuatro clases de cadena lateral

1. **No polares alifáticas:** Glicina (Gly, G), Alanina (Ala, A), Valina (Val, V), Leucina (Leu, L), Isoleucina (Ile, I), Metionina (Met, M) y Prolina (Pro, P). La glicina abre la lista por una razón: su cadena lateral es un único átomo de hidrógeno. Es el más pequeño de los veinte, el único aciral, y el que aporta flexibilidad al esqueleto allí donde cualquier otra cadena lateral estorbaría; por eso aparece en los giros cerrados y en las bisagras. La prolina está en el extremo opuesto: su cadena lateral se cierra sobre el propio nitrógeno formando un anillo, lo que le impide adoptar la mayoría de los ángulos y rompe las hélices.
2. **Aromáticas:** Fenilalanina (Phe, F), Tirosina (Tyr, Y, que lleva un hidroxilo y es por tanto anfipática) y Triptófano (Trp, W, con anillo de indol). Las tres absorben luz ultravioleta cerca de 280 nm, y esa es la razón por la que la concentración de una proteína se mide a esa longitud de onda.
3. **Polares sin carga:** Serina (Ser, S), Treonina (Thr, T), Cisteína (Cys, C), Asparagina (Asn, N) y Glutamina (Gln, Q). La cisteína merece atención aparte: su grupo tiol ($-\text{SH}$) puede oxidarse y formar un puente disulfuro covalente con otra cisteína, dentro de la misma cadena o entre cadenas distintas. Es el único enlace covalente que estabiliza la estructura terciaria, y abunda en las proteínas que trabajan fuera de la célula.
4. **Cargadas a pH fisiológico:** las ácidas, con carga negativa, son el aspartato (Asp, D) y el glutamato (Glu, E); las básicas, con carga positiva, son la lisina (Lys, K), la arginina (Arg, R) y la histidina (His, H). La histidina es un caso especial: el $\text{p}K_a$ de su anillo de imidazol ronda 6,0, muy cerca del pH fisiológico, así que puede estar protonada o desprotonada según el entorno local. Esa ambigüedad es justamente lo que la hace útil como lanzadera de protones en el centro activo de muchas enzimas.

La clasificación de aminoácidos incluye los veinte estándar; el aspartato tiene cadena lateral ácida con carga negativa y la lisina cadena básica con carga positiva a pH fisiológico.

Se reproduce con `verification/chapter_checks.py aa_class D`.

Se reproduce con `verification/chapter_checks.py aa_class K`.

Comprueba. Cuente los aminoácidos de la lista anterior antes de seguir. Si no le salen veinte, vuelva atrás: una clasificación a la que le falta un residuo es un error que se arrastra hasta el examen. La glicina es la que se pierde más a menudo, porque su cadena lateral es tan pequeña que cuesta verla como una cadena lateral.

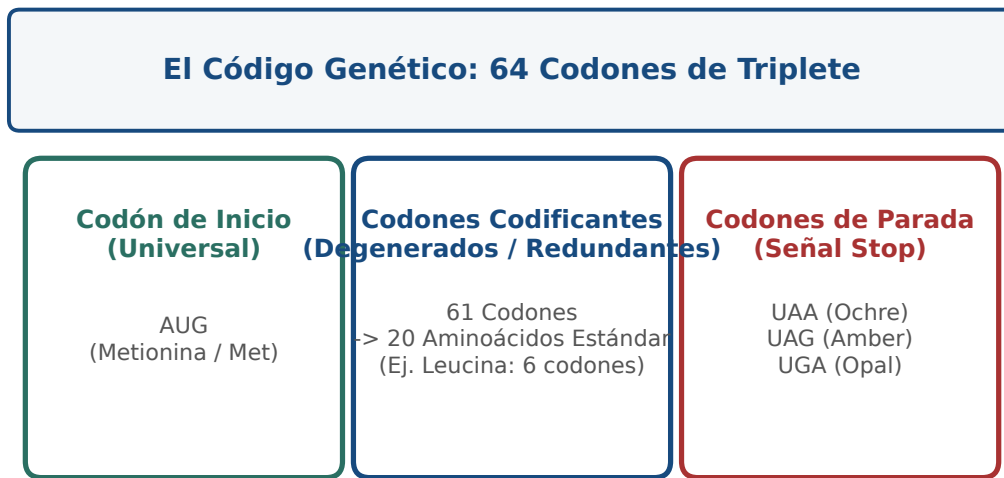


Figura 2: Los 64 codones del código genético y su redundancia. Figura original generada por `verification/make_figures.py`.

5 Carbohidratos: energía, andamio y etiqueta

ESTUDIO Los carbohidratos tienen mala fama de ser «solo azúcares», y es injusta. Químicamente son polihidroxialdehídos o polihidroxiketonas, o compuestos que al hidrolizarse los producen, y en los azúcares simples responden a la fórmula $C_n(H_2O)_n$. Hacen cuatro cosas distintas en la célula:

1. **Combustible inmediato.** La D-glucosa es el metabolito central de la glucólisis.
2. **Almacén a medio plazo.** Polímeros ramificados de glucosa: almidón en plantas, glucógeno en animales y hongos.
3. **Estructura.** Celulosa en la pared vegetal, quitina en el exoesqueleto de los artrópodos.
4. **Etiquetas de reconocimiento.** Cadenas de oligosacáridos unidas a proteínas y lípidos en la superficie celular. De ellas dependen los grupos sanguíneos del sistema AB0, buena parte del reconocimiento inmunitario y el tropismo de muchos virus, que se agarran a esos azúcares para entrar.

5.1 Cómo se nombran y por qué son casi todos D

Los monosacáridos se clasifican por el número de carbonos (triosas, tetrasas, pentosas, hexosas) y por dónde está el grupo carbonilo. Si está en el extremo de la cadena, en C1, es un aldehído y el azúcar es una **aldosa**: D-gliceraldehído, D-ribosa, D-glucosa. Si está en un carbono interior, normalmente C2, es una cetona y el azúcar es una **cetosa**: dihidroxiacetona, D-fructosa.

El criterio D/L es el mismo de Fischer que vimos en los aminoácidos, aplicado a otro carbono: un azúcar es de la **serie D** si el hidroxilo del centro quiral más alejado del carbonilo, que en las hexosas es el C5, queda a la derecha en la proyección vertical. Casi todos los monosacáridos biológicos son D, igual que casi todos los aminoácidos son L. Las dos homoquiralidades son independientes y ninguna tiene una explicación química obvia; son huellas del origen común de la vida terrestre.

5.2 El anillo, el carbono anomérico y por qué la celulosa no se come

En agua, un monosacárido de cinco o más carbonos no se queda como cadena abierta: se cierra sobre sí mismo, porque el anillo es más estable. El oxígeno del hidroxilo de C5 ataca al carbono del carbonilo y forma un enlace

hemiacetalico en las aldosas, con un anillo de seis miembros llamado **piranosa**, o **hemicetalico** en las cetosas, con un anillo de cinco llamado **furanosa**.

Al cerrarse, el carbono del carbonilo pasa de plano (sp^2) a tetraédrico (sp^3) y se convierte en un centro quiral nuevo: el **carbono anomérico**. Puede quedar de dos maneras, que se llaman anómeros α y β según hacia dónde apunte su hidroxilo. En la proyección de Haworth de un azúcar D, el anómero α lo tiene hacia abajo y el β hacia arriba.

Esa diferencia parece un detalle de dibujo. No lo es: decide las propiedades mecánicas del polímero entero.

- **Enlace $\alpha(1 \rightarrow 4)$, en almidón y glucógeno.** El ángulo que impone hace que la cadena se enrolle en una hélice abierta y flexible. El resultado son gránulos densos pero hidratados, a los que las enzimas hidrolíticas llegan sin dificultad. El glucógeno añade ramificaciones $\alpha(1 \rightarrow 6)$ cada 8 a 12 residuos, con lo que un solo gránulo ofrece cientos de extremos por los que empezar a liberar glucosa a la vez.
- **Enlace $\beta(1 \rightarrow 4)$, en celulosa.** Para acomodar la geometría β , cada glucosa gira 180° respecto a su vecina. La cadena sale recta y rígida, como una cinta. Las cintas se apilan lateralmente con redes densas de puentes de hidrógeno, expulsan el agua y forman microfibrillas cristalinas cuya resistencia a la tracción es comparable a la del acero. Y como la mayoría de los animales no producen celulasas, esa misma glucosa que en el almidón es alimento, en la celulosa es indigerible. La diferencia entre las dos moléculas es la orientación de un hidroxilo.
- **Quitina.** Polímero lineal de *N*-acetilglucosamina con el mismo enlace $\beta(1 \rightarrow 4)$, en el exoesqueleto de los artrópodos y la pared de los hongos.

REFERENCIA Entre los disacáridos conviene recordar tres. La **sacarosa**, α -D-Glc(1 $\rightarrow$ 2) β -D-Fru, tiene los dos carbonos anoméricos comprometidos en el enlace, así que no es reductora y no reacciona por ese extremo: es el azúcar que las plantas transportan por la savia, precisamente porque es químicamente inerte. La **lactosa**, β -D-Gal(1 $\rightarrow$ 4)D-Glc, sí deja libre el carbono anomérico de la glucosa y es reductora; para digerirla hace falta lactasa en el borde intestinal, y su ausencia en la edad adulta es la norma, no la excepción, en la mayoría de las poblaciones humanas. La **maltosa**, α -D-Glc(1 $\rightarrow$ 4)D-Glc, aparece como producto intermedio de la digestión del almidón.

6 Lípidos: la frontera de la célula

ESTUDIO Los lípidos son la familia rara de las cuatro. Las otras tres se definen por su monómero; los lípidos, no. Se definen por una propiedad física: se disuelven en disolventes no polares y no en agua. Bajo esa etiqueta caben moléculas muy distintas entre sí, y lo que las une es cómo se comportan frente al agua.

6.1 Ácidos grasos: por qué el aceite es líquido y la mantequilla no

Un ácido graso es una cadena de hidrocarburo con un grupo carboxilo en un extremo, casi siempre con un número par de carbonos, entre 12 y 24. La diferencia que importa es si tiene dobles enlaces o no.

- **Saturados**, como el palmítico (16:0) o el esteárico (18:0). Sin dobles enlaces, la cadena es recta y extendida, y las cadenas vecinas se apilan muy juntas maximizando las fuerzas de Van der Waals. A temperatura corporal son sólidos.
- **Insaturados**, como el oleico (18:1 *cis*-9). Cada doble enlace en configuración *cis* introduce un codo rígido de unos 30° . Ese codo impide el apilamiento regular y debilita las atracciones entre cadenas. Funden a temperatura mucho más baja y son líquidos a temperatura ambiente.

Concepto. Toda la diferencia entre una grasa animal sólida y un aceite vegetal líquido está en la forma de la cadena, no en su composición elemental. Es un buen recordatorio de algo que reaparecerá en proteínas: en biología, la geometría suele explicar más que la fórmula.

6.2 Triglicéridos: por qué la grasa almacena tanto

Un triglicérido es glicerol esterificado con tres ácidos grasos. Al no tener grupos ionizables, es completamente hidrofóbico y se almacena en los adipocitos como gotas anhidras, sin agua asociada. Su oxidación completa rinde unos 38 kJ/g, frente a unos 17 kJ/g de glúcidos y proteínas. La ventaja real es todavía mayor de lo que sugieren esas cifras: un gramo de glucógeno arrastra unos dos gramos de agua de hidratación, y la grasa no arrastra ninguna. Por eso la reserva a largo plazo de los animales es grasa y no almidón.

6.3 Fosfolípidos: la molécula que construye una frontera sola

Un fosfolípido tiene glicerol unido a dos ácidos grasos hidrofóbicos, en las posiciones *sn*-1 y *sn*-2, y a un grupo fosfato en *sn*-3 que a su vez lleva una cabeza polar: colina, etanolamina o serina. El resultado es una molécula **anfipática**, con una cabeza que quiere agua y dos colas que la rehúyen.

Puesta en agua, esa molécula no necesita ayuda para organizarse. Se autoensambla en **bicapas** cerradas de unos 4 a 5 nm de grosor, con las colas hacia dentro y las cabezas hacia fuera. Lo interesante es la razón termodinámica: no es que las colas se atraigan entre sí, es que el agua ordenada alrededor de una cola aislada tiene entropía baja, y expulsar las colas del agua libera esa entropía. La membrana se forma porque al agua le conviene, no porque a los lípidos les convenga. A ese mecanismo se le llama **efecto hidrofóbico**, y es el mismo que pliega las proteínas.

6.4 Colesterol: un amortiguador que funciona en los dos sentidos

REFERENCIA El colesterol pertenece a los esteroides, lípidos contruidos sobre cuatro anillos fusionados y rígidos. Intercalado entre los fosfolípidos de las membranas animales, hace algo que parece contradictorio: a temperatura alta, sus anillos rígidos estorban el movimiento de las colas y la membrana se vuelve menos fluida y menos permeable; a temperatura baja, se interpone entre las cadenas saturadas e impide que se empaqueten en una fase sólida, con lo que la membrana sigue siendo fluida. No fluidifica ni solidifica: amortigua en la dirección en que haga falta.

Además, el colesterol es el precursor obligado de las sales biliares, de la vitamina D y de todas las hormonas esteroideas, del cortisol a la testosterona y el estradiol. Es una molécula estructural y una materia prima a la vez.

7 Qué hace Python cuando guarda una secuencia

ESENCIAL Un cromosoma humano tiene del orden de 10^8 bases. A partir de ahí, la diferencia entre un programa que funciona y uno que agota la memoria del servidor no está en el algoritmo sino en cómo se guardan los datos. Esta sección explica lo justo del modelo de memoria de Python para poder tomar esas decisiones con criterio [8].

Los tipos primitivos de Python representan de forma natural los polímeros biológicos; en particular, las cadenas inmutables modelan secuencias que no deben alterarse.

7.1 Una variable no contiene un valor: apunta a él

En CPython, la implementación que casi todo el mundo usa, una variable no guarda un dato: guarda la dirección de un objeto que vive en el montículo (*heap*). Ese objeto, un `PyObject`, lleva al menos tres cosas: un contador de cuántas variables lo están apuntando (`ob_refcnt`), un puntero a su tipo (`ob_type`) y los datos propiamente dichos.

Cuando una variable deja de apuntar a un objeto, porque se le asigna otra cosa o porque se hace `del`, el contador baja. Si llega a cero, la memoria se libera en ese mismo instante. Solo cuando hay ciclos de referencias, objetos que se apuntan entre sí, interviene el recolector de basura cíclico (`gc`), que pasa cada cierto tiempo a buscar grupos inalcanzables.

7.2 Mutable e inmutable, y el error que esto provoca

Python reparte sus tipos en dos grupos:

- **Inmutables** (`str`, `tuple`, `int`, `bytes`): su contenido no se puede cambiar. Cualquier «modificación» crea en realidad un objeto nuevo en otra dirección.

- **Mutable** (list, dict, set, bytearray): su contenido se cambia en el sitio, sin que el objeto cambie de identidad.

Ahora viene el error. Asignar una lista a otra variable no copia nada: las dos variables apuntan al mismo objeto. A eso se le llama *aliasing*.

```
genoma_control = ['A', 'C', 'G', 'T']
genoma_mutado = genoma_control    # no copia: es el mismo objeto
genoma_mutado[1] = 'T'           # modifica los dos
print(genoma_control)             # ['A', 'T', 'G', 'T']
```

Atención. Este fallo es especialmente peligroso porque no lanza ninguna excepción. El programa sigue, los números salen, y lo que se ha corrompido es la referencia contra la que iba a comparar. Si necesita una copia, pídala explícitamente con `genoma_control.copy()` o `list(genoma_control)`; y si la secuencia no debe cambiar nunca, guárdela en un `str` o una `tuple`, que no admiten modificación y le avisarán con un error si lo intenta.

Las colecciones de Python se corresponden con usos biológicos concretos: listas para ensamblajes que crecen, tuplas para coordenadas genómicas fijas, conjuntos para alfabetos y diccionarios para tablas de codones.

7.3 Diccionarios y conjuntos: por qué buscar en ellos es gratis

Buscar un elemento en una lista obliga a recorrerla: coste $O(N)$. Los conjuntos y los diccionarios usan una tabla de dispersión, que calcula a partir del propio valor dónde debería estar, y por eso insertan, buscan y borran en tiempo constante promedio $O(1)$. Con un alfabeto de cuatro letras la diferencia es irrelevante; con un índice de millones de k -meros decide si el programa termina hoy o la semana que viene.

```
COMPLEMENTO_ADN = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
ALFABETO_ADN = {'A', 'C', 'G', 'T'}
```

7.4 Generadores: leer un cromosoma sin cargarlo

Cuando el fichero es grande, `f.read()` y `f.readlines()` son una mala idea: traen todo el contenido a memoria de golpe. Un **generador**, una función que produce valores con `yield` en lugar de devolverlos con `return`, entrega un registro cada vez y mantiene la huella de memoria constante, $O(1)$, independientemente del tamaño del fichero.

```
def leer_fasta(ruta):
    """Produce pares (cabecera, secuencia) de uno en uno."""
    cabecera = None
    trozos = []
    with open(ruta, 'r') as f:
        for linea in f:
            linea = linea.strip()
            if not linea:
                continue
            if linea.startswith('>'):
                if cabecera:
                    yield (cabecera, "".join(trozos))
                    cabecera = linea[1:]
                    trozos = []
            else:
                trozos.append(linea.upper())
    if cabecera:
        yield (cabecera, "".join(trozos))
```

Comprueba. Fíjese en las dos líneas con `yield`. La segunda, fuera del bucle, no es un descuido: sin ella se perdería el último registro del fichero, porque nadie escribe un `>` después de la última secuencia. Es el error más frecuente al escribir un lector de FASTA, y no falla ruidosamente: simplemente devuelve una secuencia menos.

Las funciones que validan su entrada y lanzan `ValueError` ante caracteres no válidos evitan que un error de datos se propague como un resultado numérico plausible.

8 Cuatro operaciones sobre secuencias

ESENCIAL. Con la química y el modelo de memoria en la mano, ya podemos escribir las cuatro operaciones que aparecerán en todos los capítulos siguientes. Cada una es corta; lo que importa es de dónde sale y qué la puede estropear.

8.1 Contenido GC

El contenido de guanina y citosina de una secuencia es la proporción de esas dos bases sobre el total. Interesa porque, como vimos, se relaciona con la estabilidad de la doble hélice, y además correlaciona con el sesgo de uso de codones y con la presencia de islas reguladoras en promotores.

Para una secuencia S de longitud N sobre el alfabeto $\Sigma = \{A, C, G, T\}$:

$$\text{GC}\%(S) = \frac{N(G) + N(C)}{N} \times 100$$

Hágalo primero a mano con $S = \text{ATGCGATCG}$, que tiene nueve bases. Los conteos son $N(A) = 2$, $N(C) = 2$, $N(G) = 3$ y $N(T) = 2$; suman nueve, que es la primera comprobación que debe hacer siempre. La suma GC es $3 + 2 = 5$, y $5/9 \times 100 = 55,555 \dots$, que redondeado a seis decimales es 55,555556 por ciento.

Para la secuencia ATGCGATCG de longitud 9 los conteos son A:2, C:2, G:3, T:2 y el contenido GC es 55,555556 por ciento.

Se reproduce con `verification/chapter_checks.py gc_content ATGCGATCG`.

Atención. Un detalle que arruina más análisis de los que parece: si la secuencia trae caracteres que no son ACGT y usted los descarta antes de contar, el denominador ya no es la longitud original. Divida siempre por la longitud de la secuencia *limpia*, la que realmente ha contado. Y deje constancia de cuántos caracteres descartó, porque si son muchos el problema no es el cálculo, es el fichero.

8.2 Complemento reverso

Aquí es donde la convención $5' \rightarrow 3'$ deja de ser un detalle. La cadena complementaria de una secuencia no se obtiene solo cambiando cada base por su pareja: como las dos hebras son antiparalelas, hay que además invertir el orden, para que el resultado quede escrito en el sentido en que todo el mundo lee.

Formalmente, si $S = s_1 s_2 \dots s_N$, su complemento reverso R cumple

$$r_i = \text{comp}(s_{N-i+1})$$

Con $S = \text{ATGCGATC}$, el proceso tiene dos pasos. Primero se complementa base a base y sale TACGCTAG . Después se invierte de extremo a extremo y sale GATCGCAT . Ese es el resultado correcto.

El complemento reverso de la secuencia ATGCGATC , escrita de 5 prima a 3 prima, es GATCGCAT .

Se reproduce con `verification/chapter_checks.py reverse_complement ATGCGATC`.

Comprueba. Antes de ejecutar nada, aplique dos veces el complemento reverso a una secuencia cualquiera en papel. ¿Qué obtiene? Si no le sale la secuencia de partida, ha olvidado la inversión o la ha aplicado dos veces. Esta propiedad, que la operación es su propia inversa, es la mejor prueba que puede escribir para su función, mejor que memorizar un resultado concreto.

Atención. La figura muestra otra fuente de errores tan frecuente que tiene nombre propio, el error de desplazamiento en uno. Las coordenadas biológicas empiezan en 1 e incluyen los dos extremos; los índices de Python empiezan en 0 y excluyen el final. La base que un fichero GFF llama posición 100 es `seq[99]` en Python. Escriba en un comentario qué convención usa cada función, y conviértalas en un único punto del programa, no en cada llamada.

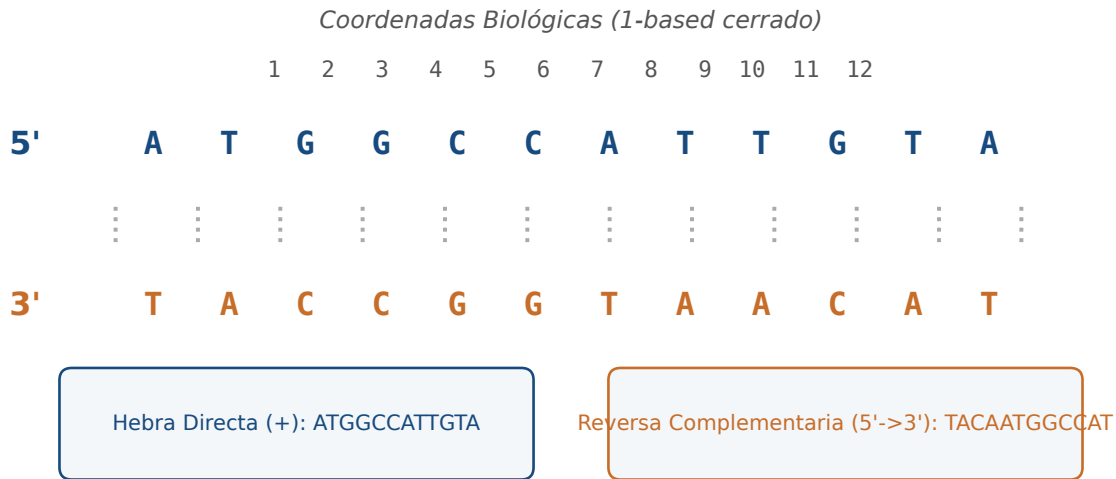


Figura 3: Orientación antiparalela del dúplex y coordenadas biológicas, que empiezan en 1 y no en 0. Figura original generada por `verification/make_figures.py`.

8.3 Transcripción

Transcribir es sustituir cada timina por un uracilo, porque el ARN usa uracilo donde el ADN usa timina:

$$\text{transcribir}(S) = S[T \mapsto U]$$

Es la operación más sencilla de las cuatro y también la más engañosa, porque el cambio de una letra oculta un cambio de molécula. El resultado ya no es ADN: no tiene sentido pedirle el complemento reverso con la tabla del ADN, ni pasárselo a una función que valide el alfabeto ACGT. Esa es exactamente la clase de descuido que la guarda de dominio del anexo está pensada para atrapar.

La transcripción sustituye timinas por uracilos y convierte ATGCGATC en AUGCGAUC.

Se reproduce con `verification/chapter_checks.py transcribe ATGCGATC`.

8.4 Ventana deslizante: el mismo resultado por dos precios

El contenido GC de un cromosoma entero dice poco: lo interesante es cómo varía a lo largo de la secuencia. Para verlo se recorre la secuencia con una **ventana** de anchura fija W que avanza de S en S posiciones, y se calcula el contenido GC dentro de cada ventana.

La forma directa de programarlo es contar las G y las C de cada ventana desde cero. Funciona, y su coste es $O(N \times W)$. Póngale números: para un cromosoma humano con $N = 10^8$ bases y una ventana de $W = 10\,000$, son del orden de 10^{12} operaciones. No es lento: es inviable.

La alternativa cuesta un poco más de pensar y mucho menos de ejecutar. Se calcula el contenido de la primera ventana una sola vez, en $O(W)$. Para cada avance, en lugar de recomtar, se ajusta: se resta la base que sale por la izquierda y se suma la que entra por la derecha.

$$GC_{k+1} = GC_k - \mathbb{I}(s_k \in \{G,C\}) + \mathbb{I}(s_{k+W} \in \{G,C\})$$

Cada paso cuesta $O(1)$ y el total es $O(W + N) = O(N)$. Los mismos 10^8 pasos que antes tardaban horas se resuelven en segundos, y el resultado es idéntico hasta el último decimal.

Concepto. Este patrón, guardar un resultado parcial y actualizarlo en lugar de recalcularlo, se llama ventana rodante y reaparece en todo el libro: en el conteo de k -meros, en el filtrado de calidad de lecturas y en los alineamientos por bandas. Cuando vea un bucle que recorre lo mismo dos veces, pregúntese qué parte del cálculo anterior podría reutilizar.

El análisis por ventana de tamaño 4 y paso 2 sobre ATGCGATCG da 0-4 ATGC con 50 por ciento, 2-6 GCGA con 75 por ciento y 4-8 GATC con 50 por ciento.

Se reproduce con `verification/chapter_checks.py sliding_window ATGCGATCG --window 4 --step 2`.

Comprueba. Con $N = 9$, $W = 4$ y $S = 2$ salen tres ventanas. ¿Sabe decir por qué no cuatro, y qué le ocurre a la última base de la secuencia? Escriba la fórmula que da el número de ventanas antes de mirar la salida del oráculo. La necesitará en el anexo.

9 Un módulo propio de secuencias

ESENCIAL Las cuatro operaciones se escriben juntas en un módulo, con anotaciones de tipo y con una guarda que valide antes de calcular. El orden importa: validar primero, contar después. Así ninguna función tiene que preguntarse si su entrada es razonable, porque ya lo comprobó la guarda. Aquí van la guarda y la primera operación completas, como modelo de lo que se espera:

```
from typing import List

ALFABETO_ADN = {'A', 'C', 'G', 'T'}
TABLA_COMPLEMENTO = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}

def validar_adn(seq: str) -> str:
    """Normaliza y valida. Devuelve la secuencia limpia o lanza ValueError."""
    limpia = seq.strip().upper()
    if not limpia:
        raise ValueError("La secuencia de ADN no puede estar vacia.")
    invalidos = set(limpia) - ALFABETO_ADN
    if invalidos:
        raise ValueError(f"Bases no validas: {sorted(invalidos)}")
    return limpia

def contenido_gc(seq: str) -> float:
    """Porcentaje de G y C sobre la secuencia ya validada."""
    s = validar_adn(seq)
    return (sum(1 for b in s if b in {'G', 'C'}) / len(s)) * 100.0
```

Las otras tres funciones las escribirá usted en el anexo. Aquí queda su contrato, que es lo que el verificador comprobará; el código es cosa suya.

- `complemento_reverso(seq: str) -> str`. Valida, complementa cada base con `TABLA_COMPLEMENTO` e invierte el orden. Aplicada dos veces a la misma secuencia debe devolver la de partida.
- `transcribir(seq: str) -> str`. Valida y sustituye cada T por una U. Conserva la longitud y no deja ninguna T en el resultado.
- `perfil_gc(seq, ventana, paso=1)`, que devuelve una lista de números en coma flotante. Valida, rechaza con `ValueError` una ventana mayor que la secuencia o un paso no positivo, y devuelve el contenido GC de cada ventana usando la actualización rodante de la sección anterior, no un recuento desde cero en cada posición. El primer valor de la lista debe coincidir con el contenido GC de las primeras ventana bases.

9.1 Validar no es desconfiar de usted: es desconfiar del fichero

Dar por bueno lo que entra es la causa más común de fallo silencioso en bioinformática. Un fichero descargado puede traer una N donde el secuenciador no supo decidir, una U porque en realidad es ARN, un espacio al final de la línea, o toda la secuencia en minúsculas porque alguien marcó así las regiones repetidas. Ninguna de esas cuatro cosas hace fallar a un programa que no valida: hacen que devuelva un número equivocado.

Por eso `validar_adn` hace dos cosas a la vez, normalizar y comprobar, y devuelve la secuencia limpia en lugar de un simple `True`. Si devolviera un booleano, quien la llame tendría que acordarse de limpiar por su cuenta, y tarde o temprano no se acordará.

La guarda de dominio rechaza con `ValueError` una secuencia que contiene bases no canónicas, como la X de `ATGCX`, y acepta las que solo contienen A, C, G y T.

10 Un genoma que no cabía en sí mismo

ESTUDIO El bacteriófago Φ X174 es un virus que infecta a *Escherichia coli* y cuyo genoma es una molécula de ADN de una sola cadena, circular, de 5386 nucleótidos. En 1977 el equipo de Fred Sanger determinó su secuencia completa con el método de terminadores dideoxi: fue el primer genoma de ADN secuenciado de la historia [4].

Y en cuanto se pudo leer, apareció un problema de contabilidad. Con 5386 bases no hay sitio para las once proteínas que el virus fabrica, si cada una ocupa su propio tramo de secuencia. Salen las cuentas mal por bastante.

La solución que usa el virus es **solapar los genes**. El gen *B* está contenido íntegramente dentro del gen *A*, leído en otro marco de lectura, y el gen *E* lo está dentro del gen *D* de la misma manera. El genoma no yuxtapone sus genes: los superpone. Una misma secuencia de bases codifica dos proteínas distintas según en qué posición empiece a leerse.

Concepto. Fíjese en lo que esto implica para nosotros. Una secuencia de ADN no tiene una única interpretación: tiene seis marcos de lectura posibles, tres en cada hebra, y cuál de ellos es el correcto no está escrito en la secuencia. Encontrar genes es, por eso, un problema de inferencia y no de lectura, y ocupará un capítulo entero más adelante. También explica por qué una mutación en un genoma tan compacto puede estropear dos proteínas a la vez: no hay bases sobrantes donde acomodar un error.

El contenido GC global de este genoma es del 44,76 por ciento, con variaciones locales que un perfil por ventana deslizante detecta y que coinciden con el origen de replicación y con los promotores tempranos. Es exactamente el análisis que usted programará en el anexo, aplicado al primer genoma que se pudo leer entero.

11 Tres errores que va a cometer

ESTUDIO La siguiente función parece razonable y hace tres cosas mal. Léala antes de seguir e intente localizarlas; las tres son errores reales, de los que aparecen en código de trabajo, no ejercicios artificiales.

```
def procesar_lecturas(lista_adn):
    limpias = lista_adn
    for i in range(len(limpias)):
        limpias[i] = limpias[i].upper()

    genoma = ""
    for seq in limpias:
        genoma = genoma + seq

    tabla = str.maketrans("ATCG", "TAGC")
    return genoma.translate(tabla)
```

1. **Modifica la lista de quien la llamó.** `limpias = lista_adn` no crea una lista nueva, sino un segundo nombre para la misma. Al escribir en `limpias[i]` se escribe en la lista original, que quizá era la referencia del análisis. La corrección es construir una lista nueva: `limpias = [s.upper() for s in lista_adn]`.
2. **Concatena en un bucle.** Como `str` es inmutable, cada `+` reserva un búfer nuevo y copia todo lo acumulado hasta ese momento. Con M fragmentos de tamaño L el coste total es $\sum_{i=1}^M i \cdot L$, es decir $O(M^2L)$. La corrección es `"".join(limpias)`, que mide una vez el total y copia una vez.
3. **Complementa sin invertir.** `translate` cambia las bases pero conserva el orden, así que devuelve la hebra complementaria escrita de 3' a 5', no el complemento reverso. La corrección es añadir `[::-1]`.

Atención. Los tres fallan en silencio. Ninguno lanza una excepción, ninguno deja el programa a medias, y los tres devuelven una cadena de la longitud correcta con las letras esperadas. Es la peor categoría de error: la que solo se detecta comparando con un resultado conocido. Por eso el método de este curso empieza siempre por predecir la salida antes de ejecutar.

El aliasing sobre listas mutables, la concatenación cuadrática de cadenas y el complemento sin inversión producen resultados incorrectos sin lanzar ninguna excepción.

12 Síntesis y tablas de diagnóstico

REFERENCIA La primera tabla recoge los fallos que más aparecen al procesar secuencias, con su causa y su corrección. Búsquela cuando algo no cuadre, antes de empezar a cambiar código a ciegas.

Síntoma observable	Causa probable	Comprobación y corrección
MemoryError al abrir un FASTA grande	se cargó el fichero entero con <code>read()</code> o <code>readlines()</code>	recorrerlo con un generador que produzca un registro cada vez
El alineamiento con la hebra reversa no puntúa	se complementó sin invertir	aplicar <code>[: :-1]</code> después de complementar; comprobar que la operación aplicada dos veces devuelve el original
El contenido GC no coincide con el esperado	se dividió por la longitud original tras descartar caracteres inválidos	dividir por la longitud de la secuencia validada
La secuencia de referencia cambió sola	<i>aliasing</i> : se pasó una lista mutable y se modificó dentro	copiar explícitamente o guardar la referencia en <code>str</code> o <code>tuple</code>
La anotación señala la base contigua a la esperada	mezcla de coordenadas 1-based cerradas con índices 0-based	convertir en un solo punto del programa y documentar la convención

La segunda tabla resume qué estructura de datos conviene a cada uso. La columna del coste de búsqueda es la que decide en cuanto los datos crecen.

Tipo	Mutabilidad	Coste de búsqueda	Uso típico en bioinformática
<code>str</code>	inmutable	$O(N)$	secuencia de referencia que no debe alterarse
<code>list</code>	mutable	$O(N)$	colección ordenada de lecturas que crece
<code>tuple</code>	inmutable	$O(N)$	coordenadas genómicas, claves compuestas
<code>set</code>	mutable	$O(1)$ promedio	alfabetos, vocabularios de k -meros
<code>dict</code>	mutable	$O(1)$ promedio	tabla de codones, índice de k -meros
<code>deque</code>	mutable	$O(1)$ en los extremos	ventanas rodantes

13 Glosario

- **Ácido nucleico:** polímero lineal de nucleótidos que se escribe de $5'$ a $3'$.
- **Enlace fosfodiéster:** unión covalente entre el $3'$ -OH de un nucleótido y el $5'$ -fosfato del siguiente.
- **Complementariedad:** apareamiento A con T por dos puentes de hidrógeno y G con C por tres.
- **Puente de hidrógeno:** interacción no covalente, mucho más débil que un enlace covalente, que puede formarse y romperse sin dañar la molécula.
- **Contenido GC:** porcentaje de guaninas y citosinas sobre el total de bases.
- **Complemento reverso:** la hebra complementaria escrita en sentido $5' \rightarrow 3'$; se complementa y después se invierte.
- **Ventana deslizante:** recorrido de una secuencia por tramos de anchura fija W separados por un paso S .
- **Ventana rodante:** implementación de lo anterior que actualiza el conteo en lugar de recalcarlo, en tiempo $O(N)$.
- **Inmutabilidad:** propiedad de un objeto cuyo contenido no puede modificarse después de crearlo.
- **Aliasing:** dos o más nombres que apuntan al mismo objeto en memoria.
- **Generador:** función que produce valores de uno

en uno con `yield` y no los guarda todos.

- **Zwitterión:** molécula con dos cargas opuestas y carga neta cero, forma habitual de un aminoácido a pH fisiológico.
- **Anómero:** cada una de las dos configuraciones, α y β , del carbono anomérico de un azúcar cíclico.

- **Efecto hidrofóbico:** tendencia de las moléculas apolares a agruparse en agua, impulsada por la entropía del disolvente.
- **Marco de lectura:** cada una de las seis formas de agrupar una secuencia en tripletes, tres por hebra.

14 Con qué sigue este capítulo

Lo que aquí se establece se da por sabido en el resto del libro. La representación de una secuencia como cadena inmutable y la guarda de dominio reaparecen en cuanto empecemos a leer formatos reales, donde los ficheros vienen sucios de verdad. El análisis de coste de la ventana rodante es el mismo razonamiento que gobierna los algoritmos de alineamiento. Y la limitación que señalamos al hablar de los priones, que la secuencia no captura el plegamiento, es el punto de partida de los capítulos de estructura.

Los contratos de inmutabilidad, validación de entrada y coste acotado establecidos en este capítulo se aplican en el resto del libro sin volver a justificarse.

15 Fuentes y reproducibilidad

La química de las cuatro familias de biomoléculas sigue a Alberts y colaboradores [1]; el modelo de ejecución de Python, al manual de referencia del lenguaje [8]; los parámetros termodinámicos de vecino más próximo, a SantaLucia [5]. Las herramientas de bioinformática en Python que este capítulo no usa, para escribir el módulo desde cero, están recogidas en Biopython [2], y las figuras se generan con Matplotlib [3] mediante `verification/make_figures.py`, con fuentes incrustadas y sin retoque manual.

Los datos de `data/` son sintéticos, de elaboración propia y con licencia CC0; su procedencia y su contenido están descritos en `data/README.md`. Todos los cálculos que aparecen en el texto se reproducen con `verification/chapter_checks.py` tal como están impresos, sin argumentos ocultos.

Anexo práctico · Escribir y verificar un módulo de secuencias

Pregunta, producto y separación de materiales

¿Puede escribir, con la biblioteca estándar de Python y sin ningún paquete externo, un módulo que calcule contenido GC, complemento reverso, transcripción y perfiles por ventana, y que rechace lo que no es ADN en lugar de devolver un número sin sentido? Este laboratorio responde que sí, y lo comprueba con secuencias que usted no ha visto.

El producto individual es un fichero, `mi_modulo_secuencias.py`, con las cinco funciones del contrato de la sección [Un módulo propio de secuencias](#). Le acompañan: la tabla de predicciones y resultados, las salidas guardadas en `resultados/`, un dictamen escrito sobre las cuatro lecturas crudas, un `README.md` con lo que ejecutó y sus supuestos, la defensa conceptual y un paquete `.tar.gz` verificado.

Este anexo es autónomo: no necesita red ni datos externos. Usa `data/` del capítulo como entrada de solo lectura y un directorio de entrega que usted crea. Los valores esperados no se imprimen aquí: los calcula usted primero a mano y los contrasta después con la herramienta que el generador deja en `herramienta/`.

Mapa de ruta y productos entregables

Bloque	Producto
1 preparar el espacio de trabajo	directorio de entrega creado sin tocar data/
2 comprobar los datos de partida	manifiesto verificado y datos inspeccionados
3 cálculo a mano de la secuencia ancla	GC y complemento reverso escritos antes de ejecutar nada
4 escribir el módulo	mi_modulo_secuencias.py con las cinco funciones
5 contrastar con el oráculo	salidas en resultados/ y filas completas en notas/respuestas.tsv
6 dictaminar las lecturas crudas	notas/lecturas.md con las cuatro decisiones
7 empaquetar y verificar	.tar.gz y verificador en ENTREGA_OK

La ruta es única. Antes de ejecutar cada orden, escriba en `notas/respuestas.tsv` lo que espera obtener. Lo que se evalúa es el contraste entre su predicción y el resultado, no el resultado solo.

1 • Preparar el espacio de trabajo

Desde el directorio del capítulo, compruebe su versión de Python y cree el directorio de entrega:

```
python3 --version
python3 verification/chapter_checks.py gc_content ATGCGATCG
```

La segunda orden reproduce el cálculo que aparece en la sección [Cuatro operaciones sobre secuencias](#). Si no coincide con lo impreso allí, deténgase: hay algo mal en su instalación antes de empezar.

```
./practice_assets/create_workspace.sh BC01_entrega
cd BC01_entrega
ls -la
```

El generador crea `datos/` con copias de los ficheros del capítulo y su manifiesto, `notas/respuestas.tsv` solo con la cabecera, `resultados/` vacía, `herramienta/` con el oráculo y el comprobador, y `mi_modulo_secuencias.py` con las cinco funciones sin escribir. Si el directorio ya existe, se niega a sobrescribirlo. No copia ninguna solución: todas las funciones de la plantilla lanzan `NotImplementedError` hasta que usted las escriba.

2 • Comprobar los datos antes de usarlos

```
cd datos && sha256sum -c manifest.sha256 && cd ..
head -n 4 datos/bc01_secuencias.fasta
cat datos/bc01_lecturas_crudas.txt
```

Comprueba. Mire las cuatro lecturas crudas y decida, antes de programar nada, qué debería hacer su guarda con cada una: aceptarla tal cual, aceptarla después de normalizarla o rechazarla. Anote las cuatro decisiones y su motivo; en el bloque 6 tendrá que defenderlas por escrito.

3 • El cálculo ancla, primero a mano

Tome la secuencia de doce bases, que es el último registro del fichero de secuencias y vale `5'-ATGCATGCGTCG-3'`, y con papel:

1. cuente cada una de las cuatro bases y compruebe que suman doce;
2. calcule el contenido GC exacto y anótelos con seis decimales;
3. escriba el complemento reverso, complementando primero e invirtiendo después;
4. escriba la transcripción a ARN;

5. con ventana $W = 6$ y paso $S = 3$, diga cuántas ventanas salen y cuál es el contenido GC de cada una.

Escriba estos cinco resultados en la columna predicción de notas/respuestas .tsv antes de continuar. No ejecute todavía nada: una predicción escrita después de ver la salida no es una predicción.

4 • Escribir el módulo

Abra `mi_modulo_secuencias.py` e implemente las cinco funciones. La guarda y el contenido GC aparecen completos en la sección [Un módulo propio de secuencias](#) y puede partir de ahí; las otras tres son suyas. Recuerde que `perfil_gc` debe actualizar el conteo al avanzar, no recontar cada ventana desde cero.

Compruebe su avance tantas veces como quiera:

```
bash herramienta/check_modulo.sh
```

El comprobador no compara con valores memorizables: prueba cinco propiedades sobre secuencias que no aparecen en el capítulo. Que el complemento reverso aplicado dos veces devuelva la secuencia de partida. Que el contenido GC de una secuencia coincida con el de su complemento reverso, y con un recuento directo. Que la transcripción conserve la longitud y no deje timinas. Que el perfil devuelva tantas ventanas como impone el paso, y que la primera coincida con el contenido GC de las primeras ventana bases. Y que la guarda rechace una N, una U, un espacio interior y la cadena vacía, aceptando en cambio una secuencia en minúsculas y devolviéndola normalizada.

Atención. Copiar el oráculo del capítulo no sirve: los nombres de sus funciones y sus tipos de retorno son distintos de los que pide el contrato. El comprobador lo detecta en la primera línea de su salida.

5 • Contrastar con el oráculo

Cuando su módulo pase las cinco propiedades, y solo entonces, contraste sus cálculos a mano con la herramienta y guarde las salidas:

```
python3 herramienta/chapter_checks.py gc_content ATGCATGCGTCG > resultados/gc_ancla.txt
python3 herramienta/chapter_checks.py reverse_complement ATGCATGCGTCG > resultados/revcomp_ancla.txt
python3 herramienta/chapter_checks.py transcribe ATGCATGCGTCG > resultados/arn_ancla.txt
python3 herramienta/chapter_checks.py sliding_window ATGCATGCGTCG --window 6 --step 3 > resultados/perfil_w6_s3.txt
cat resultados/gc_ancla.txt resultados/revcomp_ancla.txt resultados/arn_ancla.txt resultados/perfil_w6_s3.txt
```

Complete ahora las columnas resultado y coincide de notas/respuestas .tsv. Si alguna no coincide, no corrija el número: averigüe cuál de los dos cálculos está mal y por qué. Anótelos, porque esa explicación vale más que el acierto.

Comprueba. Compruebe además que su propio módulo y el oráculo dan lo mismo sobre las secuencias largas de datos/, no solo sobre la de doce bases. Una implementación puede acertar en un caso pequeño y fallar en cuanto la ventana rueda muchas veces.

6 • Dictaminar las lecturas crudas

Pase por su guarda las cuatro lecturas del fichero de lecturas crudas y compruebe qué hace con cada una. Estas dos órdenes le dan el criterio del oráculo para dos de ellas:

```
python3 herramienta/chapter_checks.py validate_dna ATGCNNATC > resultados/guarda_lect_02.txt
python3 herramienta/chapter_checks.py validate_dna ATGCGAUCG > resultados/guarda_lect_04.txt
cat resultados/guarda_lect_02.txt resultados/guarda_lect_04.txt
```

Escriba en `notas/lecturas.md` un dictamen breve para `lect_01`, `lect_02`, `lect_03` y `lect_04`: qué hace su guarda con cada una y si es lo correcto. Distinga los dos casos que no son iguales aunque lo parezcan: una N

es una posición que el secuenciador no supo leer, y una U indica que alguien ha mezclado ARN en un fichero que dice contener ADN. Rechazar las dos es defendible; tratarlas igual, no.

7 • Clases de aminoácidos

Compruebe la clase fisicoquímica de tres residuos, uno de cada extremo de la clasificación:

```
python3 herramienta/chapter_checks.py aa_class G > resultados/clases_aa.txt
python3 herramienta/chapter_checks.py aa_class D >> resultados/clases_aa.txt
python3 herramienta/chapter_checks.py aa_class K >> resultados/clases_aa.txt
cat resultados/clases_aa.txt
```

8 • Empaquetar y verificar la entrega

Escriba README.md con las órdenes que ejecutó y los supuestos que hizo, y notas/defensa.md con la defensa que se pide más abajo. Después vuelva al directorio del capítulo, empaque y verifique:

```
cd ..
tar -czf BC01_entrega.tar.gz BC01_entrega
tar -tzf BC01_entrega.tar.gz | head -n 12
sha256sum BC01_entrega.tar.gz
./practice_assets/check_entrega.sh BC01_entrega BC01_entrega.tar.gz
```

El verificador comprueba que su módulo cumple las cinco propiedades con secuencias que no ha visto, que los datos de partida no se han modificado, que la tabla de respuestas tiene al menos cinco filas completas, que resultados/ no está vacía, que el dictamen cubre las cuatro lecturas, que el README y la defensa existen con la extensión mínima, y que el paquete se extrae con el módulo dentro. Si falta algo lo enumera y termina en ENTREGA_INCOMPLETA; si está todo, en ENTREGA_OK. No puntúa: la nota la pone quien corrige.

Rúbrica de evaluación

Sobre 10 puntos, repartidos entre lo que el verificador puede comprobar y lo que solo puede valorar quien corrige.

- **3,0 puntos, el módulo funciona:** las cinco funciones cumplen el contrato, el verificador termina en ENTREGA_OK y el paquete se extrae con el producto dentro.
- **2,5 puntos, el módulo está bien escrito:** perfil_gc actualiza el conteo al avanzar en lugar de recontar; las funciones validan antes de calcular y no repiten la validación; los mensajes de ValueError dicen qué carácter sobra.
- **2,0 puntos, predicción y contraste:** las cinco filas de respuestas llevan predicción escrita antes de ejecutar; las discrepancias están explicadas, no corregidas en silencio.
- **1,5 puntos, dictamen de las lecturas crudas:** las cuatro decisiones están justificadas y distinguen el caso de la N del caso de la U.
- **1,0 punto, defensa:** concisa, correcta y apoyada en un ejemplo de su propia entrega.

Terminar en ENTREGA_OK es requisito previo, no garantía de nota: un módulo que aprueba con un perfil que recuenta desde cero cumple el contrato y pierde la mitad del segundo apartado.

Lista de autocorrección

- mis cinco predicciones estaban escritas antes de ejecutar nada;
- el complemento reverso aplicado dos veces me devuelve la secuencia original;

- `mi_perfil_gc` no vuelve a contar la ventana entera en cada paso;
- `mi_guarda` devuelve la secuencia normalizada, no un booleano;
- sé decir qué hace `mi_guarda` con una $\mathbb{N}$ y por qué no es lo mismo que una $\mathbb{U}$;
- no he modificado ningún fichero de datos/;
- las salidas que cito están guardadas en `resultados/`;
- el verificador termina en `ENTREGA_OK`.

Defensa

En `notas/defensa.md`, entre 100 y 150 palabras, explique por qué las cadenas de texto de Python son inmutables y qué le aporta esa propiedad al manipular una secuencia de referencia, frente al riesgo de compartir una lista mutable. Use un ejemplo concreto de su propia entrega, no una afirmación general: diga qué habría pasado en su código si la secuencia de partida hubiera sido una lista y una de sus funciones la hubiera modificado.

Fuentes

Los datos de `data/` son sintéticos, de elaboración propia y con licencia CC0; su contenido está descrito en `data/README.md`. El módulo se escribe solo con la biblioteca estándar [8]; las funciones equivalentes de una biblioteca establecida, para comparar después, están en `Biopython` [2]. El oráculo del capítulo está en `verification/` y el comprobador del producto, en `practice_assets/`.

Bibliografía

- [1] Bruce Alberts, Alexander Johnson, Julian Lewis, David Morgan, Martin Raff, Keith Roberts, and Peter Walter. *Molecular Biology of the Cell*. Garland Science, 6th edition, 2014. Tratado de referencia para el dogma central, estructura de ácidos nucleicos, aminoácidos y biomoléculas.
- [2] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. *Biopython: Freely Available Python Tools for Computational Molecular Biology and Bioinformatics*, volume 25. 2009.
- [3] John D. Hunter. *Matplotlib: A 2D Graphics Environment*, volume 9. 2007. Biblioteca canónica para generación de figuras y visualización científica reproducible.
- [4] F. Sanger, G. M. Air, B. G. Barrell, N. L. Brown, A. R. Coulson, J. C. Fiddes, C. A. Hutchison, P. M. Slocumbe, and M. Smith. Nucleotide sequence of bacteriophage ϕ x174 dna. *Nature*, 265:687–695, 1977.
- [5] John SantaLucia. A unified view of polymer, dumbbell, and oligonucleotide DNA nearest-neighbor thermodynamics. *Proceedings of the National Academy of Sciences*, 95(4):1460–1465, 1998.
- [6] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27:379–423, 1948.
- [7] Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.
- [8] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, 2009. Manual oficial del lenguaje Python 3.
- [9] J. D. Watson and F. H. C. Crick. Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid. *Nature*, 171:737–738, 1953.